Lab03_IsaDias

AUTHOR
Isabella Ferreira Dias 266903

Carregmento de Pacotes

```
library(readr)
library(dplyr)
```

```
Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```
library(ggplot2)
```

Laboratório 3: Dados relacionais e operações do tipo join

INSTRUÇÃO 01

Importe, utilizando o pacote readr, cada um dos três arquivos disponíveis. Os objetos resultantes devem ser chamados flights, airlines e airports.

Verifincando conteúdo do arquivo airlines.csv

```
airlines <- read_csv("airlines.csv")
```

```
Rows: 14 Columns: 2
── Column specification ────────────────────────────────────────────────────────
Delimiter: ","
chr (2): IATA_CODE, AIRLINE

ℹ Use `spec()` to retrieve the full column specification for this data.
ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
airlines
```

```
# A tibble: 14 × 2
   IATA_CODE AIRLINE
   <chr>     <chr>
 1 UA        United Air Lines Inc.
 2 AA        American Airlines Inc.
 3 US        US Airways Inc.
 4 F9        Frontier Airlines Inc.
 5 B6        JetBlue Airways
 6 OO        Skywest Airlines Inc.
 7 AS        Alaska Airlines Inc.
 8 NK        Spirit Air Lines
 9 WN        Southwest Airlines Co.
10 DL        Delta Air Lines Inc.
11 EV        Atlantic Southeast Airlines
12 HA        Hawaiian Airlines Inc.
13 MQ        American Eagle Airlines Inc.
14 VX        Virgin America
```

Para o arquivo de aeroportos, importe apenas as colunas IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE;

```
airports <- read_csv(
  "airports.csv",
  col_select = c(
    IATA_CODE,
    CITY,
    STATE,
    LATITUDE,
    LONGITUDE
  )
)
```

```
Rows: 322 Columns: 5
── Column specification ────────────────────────────────────────────────────────
Delimiter: ","
chr (3): IATA_CODE, CITY, STATE
dbl (2): LATITUDE, LONGITUDE

ℹ Use `spec()` to retrieve the full column specification for this data.
ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
airports
```

```
# A tibble: 322 × 5
   IATA_CODE CITY          STATE LATITUDE LONGITUDE
   <chr>     <chr>         <chr>    <dbl>     <dbl>
 1 ABE       Allentown     PA        40.7     -75.4
 2 ABI       Abilene       TX        32.4     -99.7
 3 ABQ       Albuquerque   NM        35.0    -107.
 4 ABR       Aberdeen      SD        45.4     -98.4
 5 ABY       Albany        GA        31.5     -84.2
```

```
 6 ACK       Nantucket     MA        41.3     -70.1
 7 ACT       Waco          TX        31.6     -97.2
 8 ACV       Arcata/Eureka CA        41.0    -124.
 9 ACY       Atlantic City NJ        39.5     -74.6
10 ADK       Adak          AK        51.9    -177.
# ℹ 312 more rows
```

#Para arquivos de voos: ##Determine, para cada parte do arquivo, as estatísticas suficientes para a determinação do atraso médio por aeroporto de destino; ##Ao finalizar a leitura do arquivo, combine as estatísticas suficientes de modo a produzir a média de atraso por aeroporto.

```
getStats <- function(input, pos) {
  input |>
    filter(
      !startsWith(DESTINATION_AIRPORT, "1"),
      !is.na(DESTINATION_AIRPORT),
      !is.na(ARRIVAL_DELAY)
    ) |>
    group_by(DESTINATION_AIRPORT) |>
    summarise(
      soma = sum(ARRIVAL_DELAY),
      n = n(),
      .groups = "drop"
    )
}

callback <- DataFrameCallback$new(getStats)

estatisticas <- read_csv_chunked(
  "flights.csv",
  callback = callback,
  chunk_size = 1000000,
  col_types = cols_only(
    DESTINATION_AIRPORT = col_character(),
    ARRIVAL_DELAY = col_double()
  )
)

flights <- estatisticas |>
  group_by(DESTINATION_AIRPORT) |>
  summarise(
    soma = sum(soma),
    n = sum(n),
    .groups = "drop"
  ) |>
  mutate(
    atraso_medio = soma / n
  )

head(flights)
```

```
# A tibble: 6 × 4
  DESTINATION_AIRPORT   soma     n atraso_medio
```

```
  <chr>                <dbl> <int>        <dbl>
1 ABE                  12874  2220         5.80
2 ABI                   9077  2224         4.08
3 ABQ                 106415 18953         5.61
4 ABR                  -2227   657        -3.39
5 ABY                   7501   864         8.68
6 ACK                   2393   487         4.91
```

```
flights <- read_csv_chunked(
  "flights.csv",
  callback = callback,
  chunk_size = 1000000,
  col_types = cols_only(
    DESTINATION_AIRPORT = col_character(),
    ARRIVAL_DELAY = col_double()
  )
)
flights
```

```
# A tibble: 1,874 × 3
   DESTINATION_AIRPORT  soma     n
   <chr>               <dbl> <int>
 1 ABE                  3413   350
 2 ABI                  5926   446
 3 ABQ                 20448  3241
 4 ABR                  -645   124
 5 ABY                  1339   169
 6 ACT                  4436   271
 7 ACV                  2748   294
 8 ACY                  9069   701
 9 ADK                  -293    19
10 ADQ                    72    59
# ℹ 1,864 more rows
```

Instrução 02

```
# Aeroportos presentes em flights, mas ausentes em airports
flights |>
  anti_join(
    airports,
    by = c("DESTINATION_AIRPORT" = "IATA_CODE")
  )|>
  select(DESTINATION_AIRPORT)
```

```
# A tibble: 0 × 1
# ℹ 1 variable: DESTINATION_AIRPORT <chr>
```

```
# Incluindo CITY e STATE em flights
flights <- flights |>
  left_join(
    airports |>
```

```
      select(IATA_CODE, CITY, STATE),
    by = c("DESTINATION_AIRPORT" = "IATA_CODE")
  )
```

INSTRUÇÃO 03

Quantidade de Aeroportos por estado

```
flights |>
  count(STATE, name = "N_AEROPORTOS", sort = TRUE)
```

```
# A tibble: 54 × 2
   STATE N_AEROPORTOS
   <chr>        <int>
 1 TX             144
 2 CA             121
 3 AK             102
 4 FL             102
 5 MI              90
 6 NY              80
 7 CO              57
 8 NC              48
 9 ND              48
10 PA              48
# ℹ 44 more rows
```

INSTRUÇÃO 04

Carregando o pacote

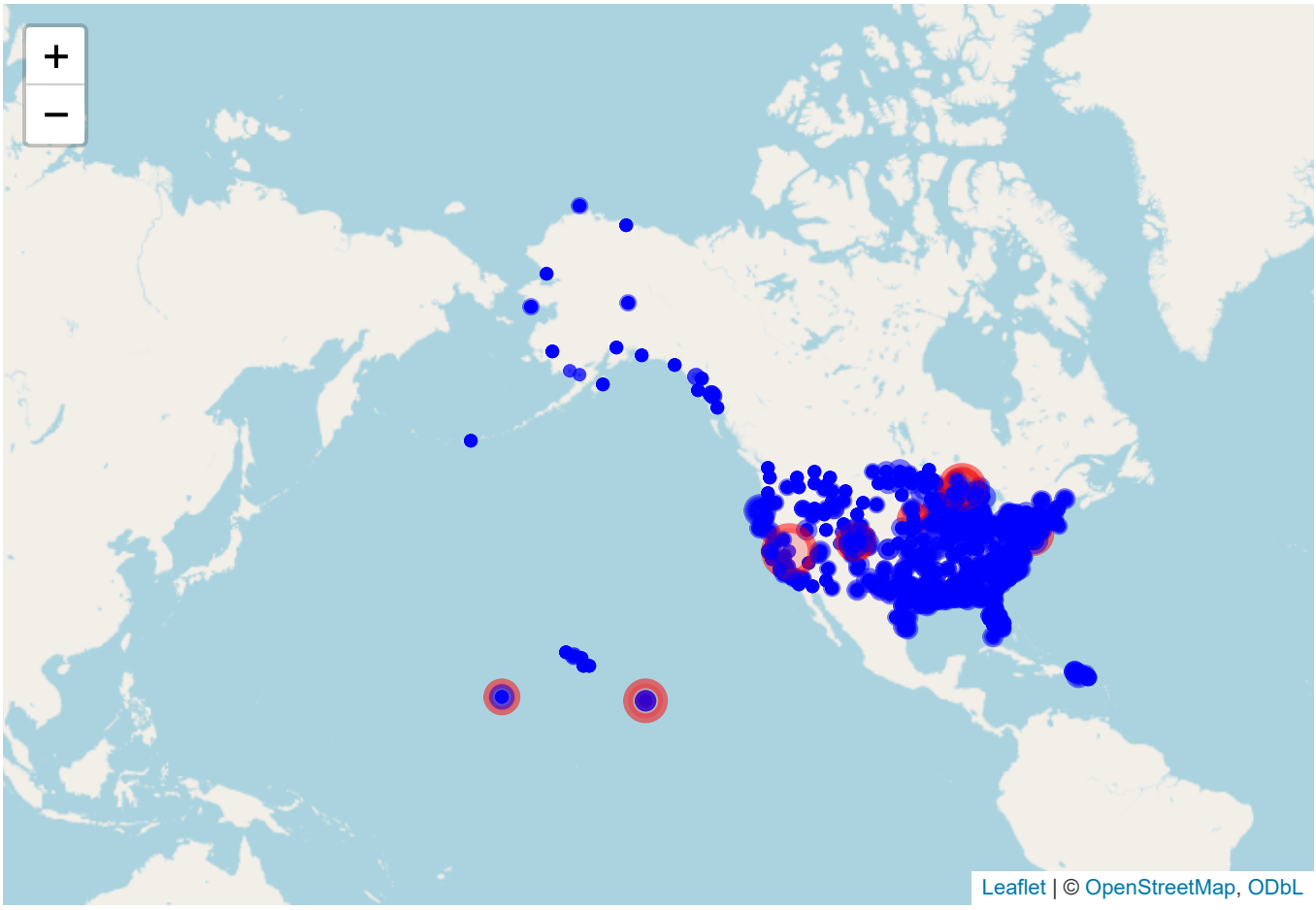
```
library(leaflet)

flights <- flights |>
  left_join(airports, by = c("DESTINATION_AIRPORT" = "IATA_CODE")) |>
  mutate(atraso_medio = soma / n)

this <- leaflet(flights) |>
  addTiles() |>
  addCircleMarkers(
    lng = ~LONGITUDE,
    lat = ~LATITUDE,
    radius = ~atraso_medio / 5,
    color = ~ifelse(atraso_medio > 30, "red", "blue"),
    popup = ~paste(DESTINATION_AIRPORT, ":", round(atraso_medio, 1), "min")
  )
```

```
Warning in validateCoords(lng, lat, funcName): Data contains 18 rows with
either missing or invalid lat/lon values and will be ignored
```

```
this
```



HORÁRIO DE COMPILAÇÃO

```
Sys.setenv(TZ = "America/Sao_Paulo")
data_hora <- format(Sys.time(), "%d/%m/%Y às %H:%M:%S")
data_hora
```

```
[1] "31/08/2026 às 20:43:16"
```